

PS 2000 B Programming Guide

Table of contents

1. Preamble.....	15
1.1 Introduction.....	15
1.2 Driver installation	15
1.2.1 Trouble-shooting driver installation problems	16
1.3 Terms	17
2. The communication protocol.....	17
2.1 Structure of the communication	17
2.2 Serial communication parameters	17
2.3 Translating set values and actual values.....	18
2.3.1 Actual values	18
2.3.2 Set values.....	18
2.4 Telegram structure	18
2.4.1 Example telegram.....	19
3. Communicating with the device	20
3.1 General.....	20
3.2 Creating a start delimiter.....	20
3.3 The mask for objects with a control byte	21
3.4 Examples	21
3.4.1 Switch device to remote mode	21
3.4.2 Query status	21
3.5 Possible problems when setting device conditions.....	22
3.6 Acknowledging and error messages.....	22
3.7 Trouble-shooting	23
3.8 Object list.....	23

1. Preamble

1.1 Introduction

This guide was written to explain the binary communication protocol for devices of series PS 2000 B to the user. The device is accessed via the built-in USB port. The USB driver will create a virtual COM port (VCP) for any new unit of this device type. Using the COM port will reduce programming of the communication port and its access to a minimum, because a COM port is normally just opened, configured, used and closed afterwards. Data is always sent to the open port.

Note: this guide does not instruct about programming and access of real or virtual hardware ports in the various programming languages. Information about this can be found in books or the internet.

1.2 Driver installation

Connect the unit to the PC via the included USB cable. Take care of fitting the mini USB plug correctly and tightly into the socket on the front (might require some force to push it in completely). If the cable is too short for the final placement of the unit, it can be extended by another USB cable (max. 5m) or an USB hub. Windows installs the driver once for every PS 2000 B that is new to the system. In case the driver is not already present on the system, it has to be installed once.

Windows should try to find and install the driver like this:

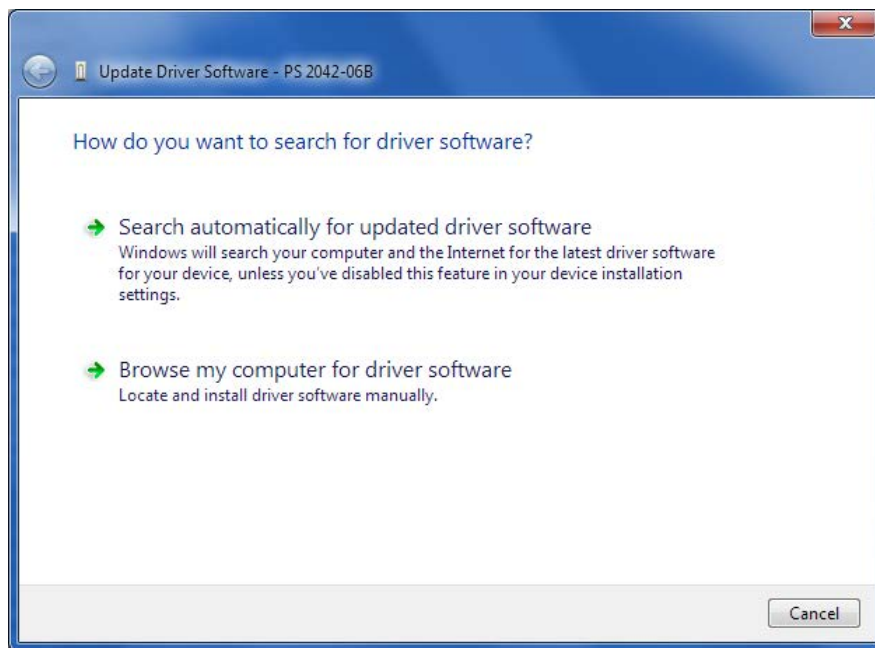


Figure 1

Always select the lowermost option here (Windows XP/Vista/7). In the next dialogue windows you are requested to point to the driver, which is located on the included CD in folder \driver. The correct installation of driver and device can be verified by opening the device manager and looking for

 PS 2000 Single (COM8) or PS 2000 Triple (COM8), depending on the model,

in section „Ports (COM & LPT)“. COM8 is just an example. If multiple units are connected to the PC, there should be multiple entries here.

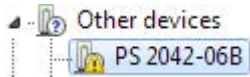
1.2.1 Trouble-shooting driver installation problems

When connecting a new device to the PC and if the driver is not already installed on the system, Windows should ask for a driver. In case it does not, the failed driver installation is reported by Windows. In this case you need to open the Windows device manager:

XP: Start (or desktop) → Right-click „Computer“ → Properties → Tab „Hardware“ → Button „Device manager“

Vista / Windows 7: Start (or desktop) → Right-click „Computer“ → Properties → Link „Device manager“ or type „device manager“ in the search bar and select it from the result list

In the section „Other devices“ you should see the device to install, like for example:



Then right-click the entry and select „Update Driver Software“. The well-known Windows driver installation dialogue will appear. Select the lowermost option (example is from Windows 7, Vista and XP are similar):

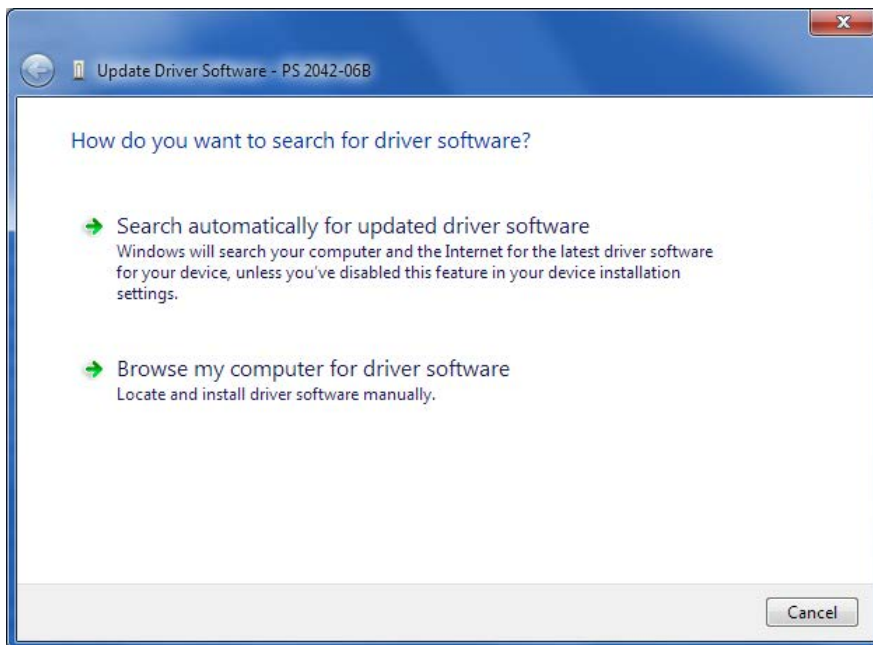


Figure 2

In the next window select the path to the driver on the EasyPS2000 CD:

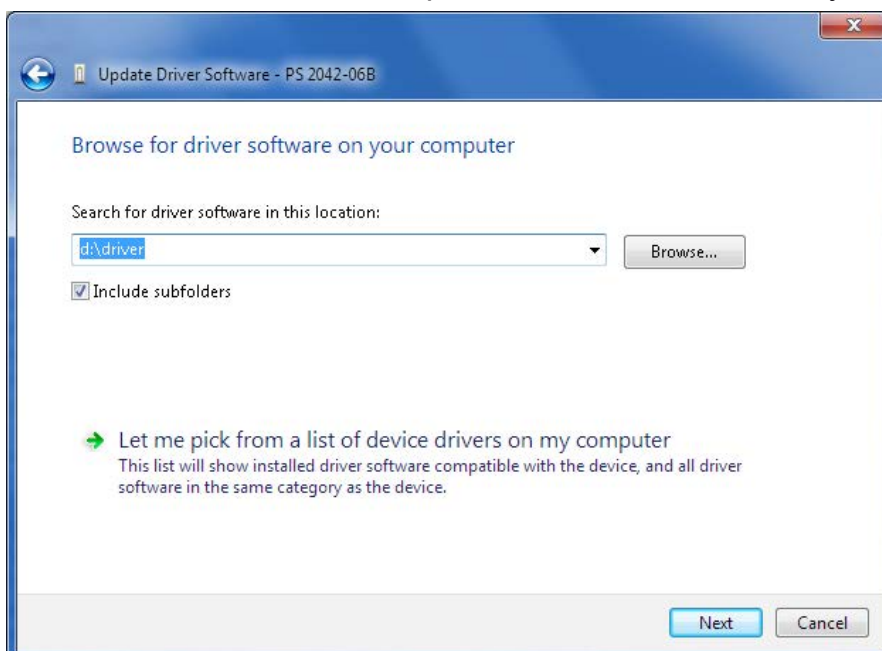
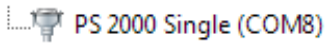


Figure 3

Windows will then install the device. After a successful installation the device can be found as „PS2000 Single“ or „Triple“ in the device manager, in section „Ports (COM & LPT)“.



Note: The COM port number 8 above is only an example. Windows assigns a new COM port for every new device of this type that is installed in the system. The port is remembered and used again if the device is connected the next time.

Note: In case there are multiple units connected to the PC, they will all be listed as single devices in the device manager.

1.3 Terms

Telegram = Chain of hexadecimal bytes, with varying length. It is either sent to a device or received from it. A part of the telegram (data field) represents hexadecimal values or ASCII strings.

2. The communication protocol

2.1 Structure of the communication

The communication with the unit is based on these telegram types:

a) Message: an object is sent which shall, for instance, set the output voltage. As long as this action is permitted by the current state of the device, the object is accepted and executed. The device will send an answer in form of an error message, containing the error code 0. If the execution of the command is not possible due to the current state of the device or the telegram is wrong, an error message with error code other than 0 is returned.

b) Query: a query is sent to the device and an answer is expected. If the query is permitted for the current state of the device it is executed and answered. The answer contains the requested data. If not permitted it will send an error message as answer.

2.2 Serial communication parameters

Data transfer is done via a virtual COM port (VCP), which is generated by the USB driver.

With the serial transmission of one byte following bits are sent:

Start bit + 8 Data bits + Parity bit + Stop bit

The parity is checked for „odd“.

Set these parameters for the particular port on the PC:

Baud rate: 115200 Bd

Parity: odd

Stop bits: 1

On the device, serial settings can not be altered.

2.3 Translating set values and actual values

Set values and actual values are transmitted as percentage values. A value of 0x6400 corresponds to 100.00%. If a device has a nominal voltage of 42V and the received actual value is 0x3200 = 50.00%, then it corresponds to 21V output voltage.

The high byte is the percentage number (0x64 = decimal 100) and the low byte is the decimal place of it, whereas 0x500 would be 5%, for example, but 0x550 would not be 5.5%. You need to translate the outgoing set values and the incoming actual values before they can be used.

2.3.1 Actual values

Actual values are queried and read from the device and will be returned as hexadecimal 16 bit values, representing a per cent value.

$$\text{Real actual value} = \frac{\text{Nom. value} * \text{Per cent act. value}}{25600}$$

Example: The nominal value of the device is 42V, the percentage actual value came in as 0x2454 = 9300. It results in: Actual value = $42 * 9300 / 25600 = 15.26\text{V}$.

2.3.2 Set values

Set values have to be translated into 16 bit per cent values before transmission. Reading set values back from the device requires to translate them vice versa.

$$\text{Percentage set value} = \frac{25600 * \text{Real set value}}{\text{Nom. value of the device}}$$

Example: the set value for voltage shall be 25.5V, the nom. value of the device is 42V. With the formula it results in: Percentage set value = $25600 * 25.5 / 42 = 15543 = 0x3CB7$. After sending 0x3CB7 to the 42V model, it should set 25.5V output voltage.

2.4 Telegram structure

The telegram is structured like this:

Byte 0	Byte 1	Byte 2	Variable bytes	2 bytes
SD	DN	OBJ	DATA	CS

and is built of these bytes:

Byte 0: **SD** (start delimiter)

The start delimiter determines how to handle the telegram furthermore. Meaning of the bits:

Bits 0-3: Data length -1 of the data in the data field of the telegram (bytes 3-18)

The data length - 1 of the data in the telegram when sending. At a query, the data length of the expected data is given here.

Bit 4: Direction

0= Telegram from device to control unit

1= Telegram from control unit to device

Bit 5

1= Cast type, for sending/querying it must be set to 1, in answers it will be 0

Bits 6+7: Transmission type

00 = Reserved

01 = Query data

10 = Answer to a query

11 = Send data

Byte 1: **DN** (device node)

Here you need to distinguish whether a Single or Triple model of PS 2000 B series is going to be controlled. With a Single model (one DC output), the DN is ignored and can be any value, but recommended to be left zero. With a Triple model (three DC outputs), following applies:

- a Triple model of PS 2000 B always has two device nodes (DN): 0 and 1
- device node 0 is assigned to DC output 1 and device node 1 is assigned to DC output 2

The control of both output regarding the objects is identical, they are only distinguished by this byte.

Byte 2: **OBJ**

The communication objects for a device are addressed by this byte. In the communication object lists, the objects and their function(s) are explained in detail. See below in section 3.8.

Byte 3 - 18: **Data field**

The data field can be 0-16 bytes long, hence the length of the telegram varies. If a query is sent (PC -> device) and no data is sent, the data field is not used and the checksum of the telegram follows directly after byte 2. Data are only transmitted when sending something to the device or when receiving an answer from it.

Word x: **CS** (check sum)

The check sum is always located at the end of the telegram. It is built by the simple addition of all preceding bytes of the telegram. It is two bytes long. The high byte is placed before the low byte.

2.4.1 Example telegram

Actual values shall be queried from the device. According to the object list, this can be done with object 71. The telegram to query the actual values has to look like this:

75 00 47 00 BC (Single model or output 1 of a triple model)

75 01 47 00 BD (Single model or output 2 of a triple model)

The answer from the device could be like this (single model or output 1 of a triple model):

85 00 47 01 01 64 00 1E 00 01 50

This will translate to 42V (green = actual voltage) and 1.8A (blue = actual current) with a PS 2042-06B with 42V nominal voltage and 6A nominal current. The status (see description of object 71) is returned as 0x0101 (pink).

3. Communicating with the device

3.1 General

It applies:

- Monitoring, i.e. only querying actual values and status, is always possible. The device doesn't require to be in remote mode in this case
- Setting of status and set values (controlling) requires the activation of the remote control mode
- With the triple models, outputs 1 and 2 are remotely controllable and have to be operated separately

In order to start controlling a device, for example by sending a set value, you need to

1. activate the remote mode (object 54)

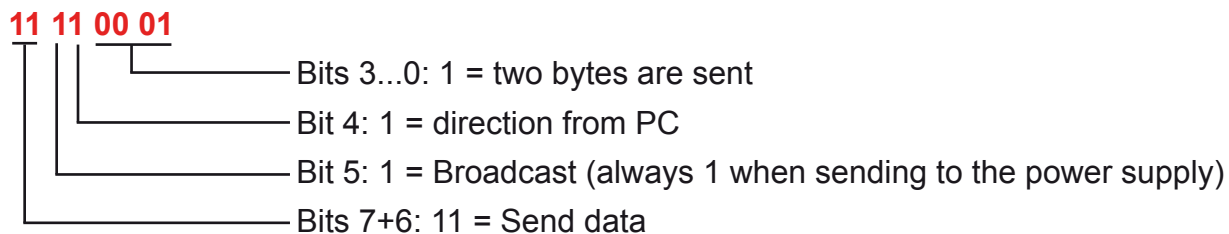
2. send the set value

3. set the output/input to on (if not already on)

Remote control should be left again, if not used any further. As long as it is active, the device can not be operated manually. The mode is indicated on the front display.

3.2 Creating a start delimiter

According to the telegram format (see above), the first byte of a telegram is the start delimiter, which depends on the type and direction of the telegram. For example, the SD can be 0xF1 and looks like this in single bits:



The SD determines that data are sent to the device. The content of the data and the object define what is sent and what the device will do in reaction.

Alternatively to the bitwise assembly, this can be simplified by adding hex values:

SD = Message type + Cast type + Direction + Length

whereas the message type is either

0xC0	Send data or
0x40	Query data
0x80	Answer from device

and the cast type is either

0x00	Answer from device (single cast) or
0x20	Broadcast (sending/querying data)

and the direction is either

0x10	from PC to the device or
0x00	from device to the PC

and the data length - 1 can be

0x01...0x0F	up to 16 bytes of data
--------------------	------------------------

By the above example, the SD of 0xF1 is built from 0xC0 + 0x20 + 0x10 + 0x01.

3.3 The mask for objects with a control byte

Some objects require a mask, which is sent together with the control byte. The possible mask value is given in column 6 of the object list (see separate PDF file or section „3.8. Object list“). The bits of the control byte have various functions, so it has to be determined which bit has to be changed. This is determined by the mask with a 1 for the corresponding bit of the control byte.

Example: bit 0 of the control byte shall be changed to 0 or 1. This will result in the control byte being either 0x00 or 0x01 and the mask being 0x01. Also see the object list, for example object 54.

3.4 Examples

3.4.1 Switch device to remote mode

The object to use is 54 (in hexadecimal 0x36), the mask for the remote mode (also see object lists) is 0x10 and the control byte for remote mode is 0x10. The telegram results as this:

F1 00 36 10 10 01 47 resp. **F1 01 36 10 10 01 48**

In order to reverse this command, deactivation of the remote mode that is, you need to send

F1 05 36 10 00 01 37. The mask stays the same, only the control byte changes.

Note: with the triple models, output 1 and 2 have to be separately set into remote control!

3.4.2 Query status

The status can either be queried with object 71 or object 72, if the other returned values (actual values or set values) can be ignored. The returned status is identical with both objects. First, you need to send the query (object 71):

75 00 47 01 38 resp. **75 01 47 01 39**

and the device will return an answer like this:

65 00 47 00 05 17 36 64 00 01 62

The first status byte 0x00 states, that the device is not in remote mode, i.e. local mode, and the second status byte states with 0x05, that the output is on and constant current mode (CC) active.

3.5 Possible problems when setting device conditions

The object 54 is used to either activate/deactivate the remote control operation or the input/output of a device. The object can be used to activate both at once, but it is strongly not recommended to do so, because setting the input/output requires the remote control operation already being active and else would generate an error message. Best way is to activate remote control first with the corresponding bit set in the control byte and then control the input/output by sending object 54 a second time with a different control byte. When deactivating remote control it simply goes vice versa.

It is also useful to read back the state of the device with object 70, in order to check if object 54 has been set correctly.

3.6 Acknowledging and error messages

When sending values or condition, the device will return an acknowledging message in form of an error message containing error code 0. This indicates that the last command was received and executed correctly. Otherwise, if the device cannot execute the last command by some reason or the telegram was wrong, an error message containing an error code other than 0 is returned. See following table:

Error code		
Hex.	Dec.	Description
00	0	No error
03	3	Check sum incorrect
04	4	Start delimiter incorrect
05	5	Wrong address for output
07	7	Object not defined
08	8	Object length incorrect
09	9	Read/Write permissions violated, no access
0F	15	Device is in "Lock" state
30	48	Upper limit of object exceeded
31	49	Lower limit of object exceeded

Legend

	Communication error
	User error

Example: if you try to set the output voltage of the device while it is not in remote control, the device returns the error message **80 00 FF 0F 01 8E**. The error code 0x0F indicates, that the device is in „local mode“, means not in remote control and won't thus accept the set value.

Explanation of some error codes:

Code 0x7: the object number used in the telegram is unknown to the device.

Code 0x8: the length of the data field in the telegram is defined in the object list. This error code will be returned if a set value, which is always 2 bytes because of type „int“, should have been sent but the data field only contained one byte. Even if the start delimiter contained the correct telegram length. This is a protection against setting wrong values.

Code 0x9: the user tried to write to an object which is, according to the object list, read only (ro).

3.7 Trouble-shooting

Problem: Multiple queries were sent, but not all of them were answered

Cause: The queries have been sent too fast. Depending on the execution time of the device, you need to include a certain latency between two transmissions. The minimum time between the transmission of two commands is defined at **50ms** for a device of PS 2000 B series.

Problem: Set values and status are not set

Possible causes

- The contacted device is not in remote control mode
- The sent values are wrong (too high, too low). An error message is returned. Or the value is accepted, but can not be transferred to the output because of the current device condition, like when sending a voltage set value while the device is in current limitation.

3.8 Object list

See external PDF file named „object_list_ps2000b_de_en.pdf“ (german/english) or „object_list_ps2000b_cn_en.pdf“ (chinese/english) for the most recent list of available communication objects. These two PDFs should always be along with this document.

